Index 3: MySQL Index Hints

EXPLAIN, USE INDEX, FORCE INDEX, IGNORE INDEX và thực nghiệm có kiểm soát

Bài giảng Database Management Systems

MySQL 8.x | SQL Indexing & Query Optimization

Tháng 11 năm 2026

Lộ trình và mục tiêu

1. Tư duy optimizer và baseline
2. USE, FORCE, IGNORE INDEX
3. Scope và optimizer hints
4. Thực nghiệm và lựa chọn bền vững

Nội dung phần học

1. Tư duy optimizer và baseline
2. USE, FORCE, IGNORE INDEX
3. Scope và optimizer hints
4. Thực nghiệm và lựa chọn bền vững

Mục tiêu học tập

- Đọc `possible_keys`, `key`, `rows`, `filtered`, `Extra`.
- Phân biệt ý nghĩa chính xác `USE INDEX`, `FORCE INDEX`, `IGNORE INDEX`.
- Áp dụng `FOR JOIN`, `FOR ORDER BY`, `FOR GROUP BY` đúng scope.
- Biết khi nào dùng hint và khi nào nên rewrite query, thiết kế index hoặc update statistics.
- Lập một protocol benchmark so sánh baseline vs hinted plans.

Nguyên tắc: không FORCE INDEX ngay

- Optimizer chọn plan bằng cost model dựa trên indexes và statistics.
- Full scan có thể tốt với bảng nhỏ hoặc filter không selective.
- Hint khóa một phần không gian lựa chọn; có thể **làm truy vấn chậm hơn**.
- Quy trình đúng: đo baseline $\rightarrow$ phân tích nguyên nhân $\rightarrow$ thử chỉnh SQL/index/stats $\rightarrow$ mới dùng hint.

Điểm cần nhớ

Tối ưu mục tiêu thực tế: latency, throughput, CPU/I/O và độ ổn định theo tham số.

Lý do optimizer không dùng index

Hiện tượng	Nguyên nhân khả dĩ
Bảng nhỏ	Table scan rẻ
`status = Shipped` nhiều row	Predicate kém selective
Index `(a,b)` nhưng filter `b`	Không khớp key prefix
`YEAR(order_date)`	Function ngăn range trực tiếp
`SELECT *` nhiều cột	Row lookup đắt
Statistics cũ / skew	Cost estimate chưa phù hợp

Chuẩn bị bốn index cho lab

```
USE index_lab;

CREATE INDEX idx_order_customer
ON order_search(customer_id);
CREATE INDEX idx_order_status
ON order_search(status);
CREATE INDEX idx_order_customer_date
ON order_search(customer_id, order_date);
CREATE INDEX idx_order_status_date
ON order_search(status, order_date);
```

Thận trọng

Chỉ tạo nếu chưa tồn tại trong schema lab. Dùng `SHOW INDEX` để tránh duplicate name, sau đó đánh giá index thừa.

Baseline query: không hint

- Ghi lại kết quả trước khi thay đổi SQL.
- Index ứng viên thường là `(customer_id, order_date)`, nhưng kế hoạch thực tế phụ thuộc dữ liệu.

```sql
EXPLAIN
SELECT order_number, order_date, status
FROM order_search
WHERE customer_id = 103
  AND order_date >= '2004-01-01'
ORDER BY order_date;
```

EXPLAIN: cách đọc từng cột

Trường	Ý nghĩa thực hành
`possible_keys`	Các index có thể dùng để tìm row
`key`	Index optimizer chọn
`key_len`	Số byte của key dùng
`rows`	Ước lượng row cần xét
`filtered`	Ước lượng % qua điều kiện
`Extra`	Using where / index / filesort...

Điểm cần nhớ

Không lấy ví dụ output giả định làm số liệu thật; chạy EXPLAIN trên server đang dùng.

Một plan giả định: đọc đúng bản chất

Minh họa, không phải output benchmark thực tế:

- `range`: có thể quét vùng key thay vì toàn table.
- `rows=12` là estimate; không có nghĩa query chạy đúng 12 row.
- Nếu `key=NULL`, trước tiên xét table size/selectivity, không vội kết luận lỗi.

```
-- Output EXPLAIN (minh hoa)
-- type          : range
-- possible_keys : idx_order_customer_date
-- key           : idx_order_customer_date
-- rows          : 12
-- Extra         : Using index condition
```

EXPLAIN ANALYZE: khác EXPLAIN

- Thực thi SELECT và báo actual rows, timing cho các operator.
- So sánh estimate với observed; tìm vị trí có lệch lớn.
- Trên hệ thống thật cần kiểm tra chi phí, quyền và độ an toàn query.

```
EXPLAIN ANALYZE
SELECT order_number, order_date, status
FROM order_search
WHERE customer_id = 103
  AND order_date >= '2004-01-01';
```

Thận trọng

Không coi `EXPLAIN ANALYZE` là lệnh chỉ đọc metadata; nó thực thi query được phân tích.

Nội dung phần học

1. Tư duy optimizer và baseline

2. USE, FORCE, IGNORE INDEX

3. Scope và optimizer hints

4. Thực nghiệm và lựa chọn bền vững

USE INDEX: giới hạn tập ứng viên

- USE INDEX nói optimizer chỉ xét các index được chỉ định cho scope tương ứng.
- Không bảo đảm index đó được dùng bằng mọi giá; vẫn có thể scan.
- Tên trong ngoặc là **index name**, không phải column name.

```
EXPLAIN
SELECT order_number, order_date, status
FROM order_search USE INDEX (idx_order_customer_date)
WHERE customer_id = 103
  AND order_date >= '2004-01-01'
ORDER BY order_date;
```

Ví dụ lỗi: dùng column name thay index name

```
-- Sai neu khong co index mang ten customer_id
SELECT * FROM order_search
USE INDEX (customer_id)
WHERE customer_id = 103;

-- Dung neu index da tao
SELECT * FROM order_search
USE INDEX (idx_order_customer)
WHERE customer_id = 103;
```

Điểm cần nhớ

Tra `SHOW INDEX FROM order_search` để xác định đúng `Key_name`; khóa chính có tên `PRIMARY`.

USE INDEX () để kiểm tra table scan

- Danh sách rỗng hạn chế optimizer dùng index cho row lookup.
- Có ích cho diagnostic: so sánh read-all vs access qua index.
- Không nên dùng mặc định trong code ứng dụng.

```
EXPLAIN SELECT order_number, order_date
FROM order_search USE INDEX ()
WHERE customer_id = 103;
```

FORCE INDEX: thiên vị index được chỉ định

- FORCE INDEX khiến optimizer coi table scan đắt khi có thể dùng index chỉ định.
- Không đảm bảo index luôn được dùng nếu không khả thi.
- Nếu index không khớp predicate, FORCE không “sửa” được cấu trúc sai.

```
EXPLAIN
SELECT order_number, order_date, status
FROM order_search FORCE INDEX (idx_order_customer_date)
WHERE customer_id = 103
  AND order_date >= '2004-01-01'
ORDER BY order_date;
```

USE vs FORCE: phân biệt chính xác

Tiêu chí	USE INDEX	FORCE INDEX
Ý nghĩa	Giới hạn ứng viên	Giới hạn + làm full scan kém hấp dẫn
Scan table	Vẫn dễ được xét	Bị tăng chi phí trong model
Có chắc dùng key?	Không	Không tuyệt đối
Rủi ro	Chặn index tốt hơn	Ép plan không tối ưu

Điểm cần nhớ

So sánh `EXPLAIN` và actual timing. Tên “FORCE” không có nghĩa cưỡng chế bất chấp mọi tình huống.

Phản ví dụ: ép index sai

- `status` không phải leftmost predicate của query.
- Tạo/sửa index và rewrite query thường tốt hơn cưỡng ép một key không phù hợp.

```
-- Query loc customer/date
SELECT * FROM order_search
FORCE INDEX (idx_order_status_date)
WHERE customer_id = 103
  AND order_date >= '2004-01-01';

-- Can nhac index dung pattern
CREATE INDEX idx_order_customer_date
ON order_search(customer_id, order_date);
```

IGNORE INDEX: loại một ứng viên để thử

- Có thể so sánh baseline với plan khi index cụ thể bị loại.
- Hữu ích kiểm tra index có thực sự giúp hay không.
- Nếu có index khác, optimizer có thể chuyển sang index khác thay vì full scan.

```
EXPLAIN
SELECT order_number, order_date
FROM order_search IGNORE INDEX (idx_order_customer_date)
WHERE customer_id = 103
  AND order_date >= '2004-01-01';
```

So sánh bốn biến thể cùng một query

Biến thể	Mục đích kiểm tra
Không hint	Baseline optimizer tự chọn
`USE INDEX (idx_order_customer_date)`	Chỉ xét key được nêu trong scope
`FORCE INDEX (idx_order_customer_date)`	Giảm hấp dẫn của scan
`IGNORE INDEX (idx_order_customer_date)`	Quan sát plan khi bỏ một key

Điểm cần nhớ

Mọi so sánh phải giữ dataset, indexes khác, parameter values và trạng thái cache càng tương đồng càng tốt.

Nội dung phần học

1. Tư duy optimizer và baseline
2. USE, FORCE, IGNORE INDEX
3. Scope và optimizer hints
4. Thực nghiệm và lựa chọn bền vững

FOR JOIN: scope truy cập row

- `FOR JOIN` nói về row lookup / join planning.
- Không yêu cầu query phải có từ khóa JOIN để scope này có ý nghĩa.

```
EXPLAIN SELECT order_number, order_date
FROM order_search
USE INDEX FOR JOIN (idx_order_customer_date)
WHERE customer_id = 103
  AND order_date >= '2004-01-01';
```

FOR ORDER BY: scope cho thứ tự

- Index chỉ có thể hỗ trợ thứ tự khi query plan giữ được key order.
- Có hint không bảo đảm biến mất `Using filesort`.

```
EXPLAIN SELECT order_number, status, order_date
FROM order_search
USE INDEX FOR ORDER BY (idx_order_status_date)
ORDER BY status, order_date;
```

FOR GROUP BY: scope cho grouping

- Xem optimizer có thể grouping thuận theo index không.
- Không có quy tắc mỗi GROUP BY đều phải index; phân tích số group, distribution và actual timing.

```
EXPLAIN SELECT status, COUNT(*) AS total_orders
FROM order_search
USE INDEX FOR GROUP BY (idx_order_status)
GROUP BY status;
```

Không trộn USE và FORCE cùng scope

```
-- Sai: USE va FORCE trung scope FOR JOIN
SELECT * FROM order_search
USE INDEX FOR JOIN (idx_order_customer)
FORCE INDEX FOR JOIN (idx_order_customer_date)
WHERE customer_id = 103;

-- Chon mot cho FOR JOIN
SELECT * FROM order_search
USE INDEX FOR JOIN (idx_order_customer_date)
WHERE customer_id = 103;
```

Điểm cần nhớ

Phân biệt scope và kiểm tra cú pháp trên phiên bản MySQL sử dụng.

Optimizer hints comment-style hiện đại

- Một số version MySQL hỗ trợ `INDEX`, `JOIN_INDEX`, `ORDER_INDEX`, `GROUP_INDEX` và NO_* variants.
- Tài liệu MySQL 8.4 định hướng thay thế dần legacy index hints.
- Tên alias table trong hint phải khớp query block.

```
EXPLAIN
SELECT /*+ INDEX(o idx_order_customer_date) */
       o.order_number, o.order_date
FROM order_search AS o
WHERE o.customer_id = 103
  AND o.order_date >= '2004-01-01';
```

Thận trọng: Đọc tài liệu đúng server version trước khi triển khai; hint có thể không hợp lệ hoặc bị bỏ qua ở version cũ.

Nội dung phần học

1. Tư duy optimizer và baseline
2. USE, FORCE, IGNORE INDEX
3. Scope và optimizer hints
4. Thực nghiệm và lựa chọn bền vững

Cập nhật statistics trước khi hint

- Data distribution có thể thay đổi sau bulk load hoặc update lớn.
- Optimizer cần ước lượng phù hợp để so sánh các paths.
- Làm mới stats xong vẫn cần EXPLAIN và benchmark; không có bảo đảm cải thiện.

```
ANALYZE TABLE order_search;

EXPLAIN SELECT order_number, order_date
FROM order_search
WHERE customer_id = 103
  AND order_date >= '2004-01-01';
```

Rewrite query thay vì ép index

```sql
-- Khong than thien voi index ngay
SELECT * FROM order_search
WHERE YEAR(order_date) = 2004
  AND customer_id = 103;

-- Range rewrite
SELECT * FROM order_search
WHERE order_date >= '2004-01-01'
  AND order_date < '2005-01-01'
  AND customer_id = 103;
```

Điểm cần nhớ

Chọn index `(customer_id, order_date)` phù hợp rồi đo; hint chỉ là bước sau nếu cần.

Khi SELECT * phá lợi ích index

- Nếu lấy rất nhiều row, index + row lookup có thể đắt hơn scan.
- Chọn đúng cột cần thiết có thể hỗ trợ covering index.
- Không sửa semantics của query chỉ vì muốn dùng index; phải đúng yêu cầu nghiệp vụ.

```sql
SELECT * FROM order_search
WHERE status = 'Shipped';

-- Hoi: thuc su can cac column nao?
SELECT order_number, status, order_date
FROM order_search
WHERE status = 'Shipped';
```

Thiết kế benchmark có kiểm soát

1. Giữ nguyên query semantics, schema, data distribution, server version.
2. Đo baseline: `EXPLAIN`, timing và resource metrics.
3. Thử 3–5 giá trị tham số đại diện (phổ biến/hiếm).
4. Thực hiện nhiều runs; phân biệt cache warm/cold nếu phù hợp.
5. Thử hint, đối chiếu plan và latency; lưu cả trường hợp xấu.
6. Gỡ hint nếu không có cải thiện ổn định hoặc gây regression.

Bảng ghi nhận kết quả thực nghiệm

- Không điền timing giả định vào báo cáo.
- Dùng nhiều input values, cân nhắc median/p95 và độ lệch giữa các runs.

Variant	**Key**	**Est. rows**	**Actual ms**
Baseline	(điền từ EXPLAIN)	…	…
USE INDEX	…	…	…
FORCE INDEX	…	…	…
IGNORE INDEX	…	…	…

Case A: query theo customer và date

1. Chạy baseline EXPLAIN.
2. Thử USE INDEX trên composite `idx_order_customer_date`.
3. Thử FORCE INDEX cùng key.
4. So sánh plan/timing; giải thích vì sao force có thể không thêm lợi ích.

```sql
SELECT order_number, status, order_date
FROM order_search
WHERE customer_id = 103
  AND order_date >= '2004-01-01'
ORDER BY order_date;
```

Case B: GROUP BY và sort

1. Lựa chọn scope `FOR GROUP BY` và `FOR ORDER BY`.
2. Kiểm tra `Using filesort`/scan và timing.
3. Giải thích vì sao một index có thể phù hợp query này nhưng không phù hợp query khác.

```sql
SELECT status, COUNT(*) AS n
FROM order_search
GROUP BY status;

SELECT order_number, status, order_date
FROM order_search
ORDER BY status, order_date;
```

Case C: sau khi thay đổi phân phối

Giả sử 95% đơn ở trạng thái `Shipped`, nhưng một số trạng thái còn lại rất hiếm.

- FORCE INDEX trên `status` cho `Shipped` có thể gây rất nhiều row lookup.
- Cùng hint trên status hiếm có thể khác kết quả.
- Plan tốt hôm nay có thể không tốt sau khi dữ liệu tăng hoặc phân phối dịch chuyển.
- Lập lịch review index hint trong application code.

Checklist trước khi đưa hint vào production

- Có dữ liệu benchmark nhiều input và baseline rõ ràng.
- Tài liệu hóa lý do hint, index name, SQL pattern, MySQL version.
- Có test regression trên dữ liệu/khối lượng tương đương.
- Theo dõi p95 latency, CPU/I/O và throughput sau triển khai.
- Có tiêu chí bỏ hint khi optimizer / statistics / workload thay đổi.

Thận trọng

Hint là ngoại lệ có quản lý, không phải biện pháp thay thế thiết kế index và query tốt.

Bài tập cuối bài: chữa một query chậm

Query cần lọc `customer_id=103` và năm 2004 nhưng đang dùng `YEAR(order_date)`.

1. Rewrite sang điều kiện range nửa mở.
2. Thiết kế index composite phù hợp.
3. Chạy EXPLAIN không hint.
4. Thử USE, FORCE rồi IGNORE cùng key và so sánh.
5. Viết 5–8 câu kết luận có nêu limitations của small dataset.

Tổng kết Index 3

- **EXPLAIN trước, hint sau**; so sánh actual timing khi có điều kiện.
- **USE INDEX** thu hẹp ứng viên; **FORCE INDEX** làm scan kém hấp dẫn; **IGNORE INDEX** loại ứng viên.
- **Scope** FOR JOIN/ORDER BY/GROUP BY gắn với giai đoạn tối ưu tương ứng.
- **Root-cause first:** rewrite, thiết kế composite index, cập nhật statistics.
- **Production:** benchmark đa tham số, kiểm soát version, giám sát và review hint định kỳ.

Điểm cần nhớ

Tham khảo: vudmvn.github.io/dbms-course → Index Optimization → Index 3.